Project Flow & Architecture Theory - Residentia (PG Finder)

Complete Interview Guide - Request Flow from Frontend to Backend

---

🎯 Project Overview

Residentia is a full-stack **PG (Paying Guest) Finder** platform where:

- **Clients** browse and book PG accommodations
- **Owners** list and manage their properties
- **Admins** oversee the entire platform

🏗️ Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + Vite	User interface, SPA
Routing	React Router v6	Client-side navigation
API Client	Axios	HTTP requests to backend
Backend	Spring Boot 3.2	REST API server
Security	Spring Security + JWT	Authentication & Authorization
Database	MySQL 8 (AWS RDS)	Data persistence
ORM	Spring Data JPA (Hibernate)	Database interaction
File Storage	Cloudinary	Property images
Payment	Razorpay Gateway	Online payments
Email	Spring Mail (Gmail SMTP)	Notifications
Logging	SLF4J + Logback	Application logs

🌐 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                      CLIENT BROWSER                          │
│                 (React SPA - Port 5173)                      │
│                                                              │
│  React Components → React Router → Axios Interceptor          │
│  (JWT token automatically added to every request)            │
└──────────────────────────────────────────────────────────────┘
                              │
                              │
```

```
                          │ HTTP/HTTPS (REST API - JSON)
                          │
                          ▼
┌──────────────────────────────────────────────────────────────────┐
│              SPRING BOOT BACKEND                                 │
│                 (Port 8080 - AWS EC2)                            │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │           Spring Security Filter Chain                   │   │
│  │  CORS → JWT Authentication → Authorization               │   │
│  └──────────────────────────┬───────────────────────────────┘   │
│                             │                                    │
│                             ▼                                    │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │              DispatcherServlet                           │   │
│  │          (Routes requests to controllers)                │   │
│  └──────────────────────────┬───────────────────────────────┘   │
│                             │                                    │
│                             ▼                                    │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │              @RestController Layer                       │   │
│  │  AuthController, OwnerController, ClientController...    │   │
│  └──────────────────────────┬───────────────────────────────┘   │
│                             │                                    │
│                             ▼                                    │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │              @Service Layer                              │   │
│  │  Business logic, validation, authentication              │   │
│  └──────────────────────────┬───────────────────────────────┘   │
│                             │                                    │
│                             ▼                                    │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │              @Repository Layer                           │   │
│  │  Spring Data JPA - Database queries                      │   │
│  └──────────────────────────┬───────────────────────────────┘   │
└─────────────────────────────┼────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│              MYSQL DATABASE (AWS RDS)                            │
│  Tables: owners, regular_users, properties, bookings...          │
└──────────────────────────────────────────────────────────────────┘
```

🔄 Complete Request Flow - Example Scenarios

SCENARIO 1: Owner Login Flow

Step-by-Step Request Journey

```
┌─────────────────────────────────────────────────────────────┐
│  STEP 1: User Fills Login Form                              │
│  Component: OwnerLogin.jsx                                  │
│  URL: http://localhost:5173/auth/owner/login                │
│                                                             │
│  User enters:                                               │
│  - Email: owner@example.com                                 │
│  - Password: password123                                    │
│  - Clicks "Login" button                                    │
└─────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌─────────────────────────────────────────────────────────────┐
│  STEP 2: React Component Calls API                          │
│  File: src/api/ownerApi.js                                  │
│                                                             │
│  const loginOwner = async (email, password) => {            │
│      const response = await axios.post(                     │
│          `${BASE_URL}/api/auth/login`,                      │
│          { email, password, role: "OWNER" }                 │
│      );                                                     │
│      return response.data;                                  │
│  };                                                         │
└─────────────────────────────────────────────────────────────┘
                    │
                    │
                    │ HTTP POST Request
                    │ http://localhost:8080/api/auth/login
                    │ Body: {email, password, role}
                    │
                    ▼
┌─────────────────────────────────────────────────────────────┐
│  STEP 3: Request Enters Spring Boot Application             │
│  Port: 8080                                                 │
│                                                             │
│  Tomcat Server receives request                             │
└─────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌─────────────────────────────────────────────────────────────┐
│  STEP 4: Spring Security Filter Chain                       │
│                                                             │
│   ┌──────────────────────────────────────────────────┐      │
│   │  Filter 1: CORS Filter                           │      │
│   │  ✓ Checks if origin allowed (localhost:5173)     │      │
│   │  ✓ Adds CORS headers to response                 │      │
│   └──────────────────────────────────────────────────┘      │
│                    │                                        │
│                    │                                        │
│   ┌────────────────▼─────────────────────────────────┐      │
│   │  Filter 2: JwtAuthenticationFilter               │      │
│   │  ⚠ No token present (login endpoint)             │      │
│   │  → Skip authentication (public endpoint)         │      │
│   └──────────────────────────────────────────────────┘      │
│                    │                                        │
│                    │                                        │
```

```
│   ┌──────────────────▼──────────────────────────────────┐   │
│   │  Filter 3: AuthorizationFilter                      │   │
│   │  ✓ /api/auth/login is permitAll()                   │   │
│   │  → Allow request to proceed                         │   │
│   └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 5: DispatcherServlet Routes to Controller              │
│                                                              │
│  Matches request to:                                         │
│  POST /api/auth/login → AuthController.login()               │
└──────────────────────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 6: AuthController Executes                             │
│  File: com.residentia.controller.AuthController.java         │
│                                                              │
│  @PostMapping("/login")                                      │
│  public ResponseEntity<?> login(@RequestBody LoginRequest) {│
│                                                              │
│      1. Extract email, password, role from request           │
│      2. Call ownerRepository.findByEmail(email)              │
└──────────────────────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 7: Repository Layer Queries Database                   │
│  File: com.residentia.repository.OwnerRepository.java        │
│                                                              │
│  Optional<Owner> findByEmail(String email);                  │
│                                                              │
│  → Hibernate generates SQL:                                  │
│    SELECT * FROM owners WHERE email = 'owner@example.com'    │
└──────────────────────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 8: MySQL Database Returns Result                       │
│  AWS RDS Instance                                            │
│                                                              │
│  Returns Owner entity with:                                  │
│  - id, email, name, passwordHash, isActive, etc.             │
└──────────────────────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 9: AuthController Validates Password                   │
│                                                              │
│  3. passwordEncoder.matches(password, owner.getPasswordHash())│
│     → BCrypt compares plain password with hashed password    │
│  4. Check if owner.getIsActive() == true                     │
```

```
│  5. If valid, generate JWT token                                  │
└──────────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────┐
│  STEP 10: JWT Token Generation                                    │
│  File: com.residentia.security.JwtTokenProvider.java              │
│                                                                   │
│  String token = jwtTokenProvider.generateToken(                   │
│      owner.getId(),                                               │
│      owner.getEmail(),                                            │
│      "OWNER"                                                      │
│  );                                                               │
│                                                                   │
│  Token contains:                                                  │
│  - userId: 123                                                    │
│  - email: owner@example.com                                       │
│  - role: OWNER                                                    │
│  - expiration: 24 hours from now                                  │
└──────────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────┐
│  STEP 11: Response Sent to Frontend                               │
│                                                                   │
│  HTTP 200 OK                                                      │
│  Body:                                                            │
│  {                                                                │
│    "token": "eyJhbGciOiJIUzUxMiJ9.eyJzdWIi...",                   │
│    "userId": 123,                                                 │
│    "email": "owner@example.com",                                  │
│    "name": "John Doe",                                            │
│    "role": "OWNER",                                               │
│    "message": "Owner login successful!"                           │
│  }                                                                │
└──────────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────┐
│  STEP 12: Frontend Stores Token                                   │
│  File: src/api/ownerApi.js                                        │
│                                                                   │
│  if (response.success) {                                          │
│      localStorage.setItem('token', response.token);               │
│      localStorage.setItem('userRole', response.role);             │
│      localStorage.setItem('userName', response.name);             │
│      navigate('/owner/dashboard');                                │
│  }                                                                │
└──────────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────┐
│  STEP 13: User Redirected to Dashboard                            │
│  URL: http://localhost:5173/owner/dashboard                       │
```

```
│                                                                │
│  Now all subsequent API calls will include the JWT token       │
└────────────────────────────────────────────────────────────────┘
```

SCENARIO 2: Owner Fetching Properties (Authenticated Request)

```
┌────────────────────────────────────────────────────────────────┐
│  STEP 1: Owner Dashboard Loads                                 │
│  Component: Owner.jsx                                          │
│  URL: http://localhost:5173/owner/dashboard                    │
│                                                                │
│  useEffect(() => {                                             │
│      fetchOwnerProperties();                                   │
│  }, []);                                                       │
└────────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌────────────────────────────────────────────────────────────────┐
│  STEP 2: Axios API Call with Token                             │
│  File: src/api/ownerApi.js                                     │
│                                                                │
│  const getOwnerProperties = async () => {                      │
│      const response = await axios.get(                         │
│          `${BASE_URL}/api/owner/properties`                    │
│      );                                                        │
│      return response.data;                                     │
│  };                                                            │
│                                                                │
│  Axios Interceptor (src/api/axios.js) automatically adds:      │
│  Headers: {                                                    │
│      Authorization: "Bearer eyJhbGciOiJIUzUxMiJ9..."           │
│  }                                                             │
└────────────────────────────────────────────────────────────────┘
                    │
                    │
                    │ HTTP GET Request
                    │ http://localhost:8080/api/owner/properties
                    │ Headers: Authorization: Bearer <token>
                    │
                    ▼
┌────────────────────────────────────────────────────────────────┐
│  STEP 3: Request Enters Backend                                │
│  Spring Boot Application (Port 8080)                           │
└────────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌────────────────────────────────────────────────────────────────┐
│  STEP 4: Spring Security Filter Chain                          │
│                                                                │
│   ┌──────────────────────────────────────────────────────┐     │
│   │  Filter 1: CORS Filter                               │     │
```

```
│   │  ✓ Origin check passed                                  │   │
│   └──────────────────┬──────────────────────────────────────┘   │
│                      │                                          │
│   ┌──────────────────▼──────────────────────────────────────┐   │
│   │  Filter 2: JwtAuthenticationFilter                       │   │
│   │                                                          │   │
│   │  1. Extract token from "Authorization" header           │   │
│   │  2. Validate token signature & expiration               │   │
│   │  3. Extract claims (email, role, userId)                │   │
│   │     - email: owner@example.com                          │   │
│   │     - role: OWNER                                       │   │
│   │     - userId: 123                                       │   │
│   │  4. Create Authentication object                        │   │
│   │  5. Store in SecurityContextHolder                      │   │
│   │  6. Set request attributes:                             │   │
│   │     request.setAttribute("email", "owner@...")          │   │
│   │     request.setAttribute("ownerId", 123)                │   │
│   └──────────────────┬──────────────────────────────────────┘   │
│                      │                                          │
│   ┌──────────────────▼──────────────────────────────────────┐   │
│   │  Filter 3: AuthorizationFilter                           │   │
│   │  ✓ /api/owner/** requires authenticated()               │   │
│   │  ✓ User is authenticated with ROLE_OWNER                │   │
│   │  → Allow request                                        │   │
│   └──────────────────┬──────────────────────────────────────┘   │
└──────────────────────┴──────────────────────────────────────────┘
                       │
                       ▼

┌──────────────────────────────────────────────────────────────────┐
│  STEP 5: DispatcherServlet Routes to Controller                  │
│  GET /api/owner/properties → OwnerController                     │
└──────────────────────┬──────────────────────────────────────────┘
                       │
                       ▼

┌──────────────────────────────────────────────────────────────────┐
│  STEP 6: OwnerController Executes                                │
│  File: com.residentia.controller.OwnerController.java            │
│                                                                  │
│  @GetMapping("/properties")                                      │
│  public ResponseEntity<List<PropertyDTO>> getProperties(         │
│      HttpServletRequest request                                  │
│  ) {                                                             │
│      // Extract ownerId from request attributes                  │
│      Long ownerId = (Long) request.getAttribute("ownerId");      │
│                                                                  │
│      // Call service layer                                       │
│      List<PropertyDTO> properties =                              │
│          propertyService.getPropertiesByOwnerId(ownerId);       │
│                                                                  │
│      return ResponseEntity.ok(properties);                       │
│  }                                                               │
└──────────────────────┬──────────────────────────────────────────┘
                       │
                       ▼
```

```
┌──────────────────────────────────────────────────────────────┐
│  STEP 7: Service Layer (Business Logic)                      │
│  File: com.residentia.service.PropertyService.java           │
│                                                              │
│  public List<PropertyDTO> getPropertiesByOwnerId(Long id) {  │
│      List<Property> properties =                             │
│          propertyRepository.findByOwnerId(id);               │
│                                                              │
│      // Convert entities to DTOs                             │
│      return properties.stream()                              │
│          .map(this::convertToDTO)                            │
│          .collect(Collectors.toList());                      │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
                   │
                   │
                   ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 8: Repository Layer - Database Query                   │
│  File: com.residentia.repository.PropertyRepository.java     │
│                                                              │
│  List<Property> findByOwnerId(Long ownerId);                 │
│                                                              │
│  → Hibernate SQL:                                            │
│    SELECT * FROM properties WHERE owner_id = 123             │
└──────────────────────────────────────────────────────────────┘
                   │
                   │
                   ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 9: MySQL Database Returns Results                      │
│  List of Property entities with all fields                   │
└──────────────────────────────────────────────────────────────┘
                   │
                   │
                   ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 10: Service Converts to DTOs                           │
│  Property Entity → PropertyDTO (expose only needed fields)   │
└──────────────────────────────────────────────────────────────┘
                   │
                   │
                   ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 11: Controller Returns Response                        │
│  HTTP 200 OK                                                 │
│  Body: [                                                     │
│      {                                                       │
│          id: 1,                                              │
│          name: "Sunshine PG",                                │
│          address: "123 Main St",                             │
│          rent: 8000,                                         │
│          images: ["url1", "url2"],                           │
│          amenities: ["WiFi", "AC"]                           │
│      },                                                      │
│      ...                                                     │
│  ]                                                           │
└──────────────────────────────────────────────────────────────┘
                   │
```

```
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 12: Frontend Receives Data                             │
│  File: Owner.jsx                                             │
│                                                              │
│  setProperties(response.data);                               │
│                                                              │
│  React re-renders with property list                         │
└──────────────────────────────────────────────────────────────┘
```

SCENARIO 3: Client Browsing Properties (Public Endpoint)

```
┌──────────────────────────────────────────────────────────────┐
│  STEP 1: Client Visits Public Browse Page                    │
│  Component: PublicBrowse.jsx                                 │
│  URL: http://localhost:5173/browse                           │
│                                                              │
│  useEffect(() => {                                           │
│      fetchProperties();                                      │
│  }, []);                                                     │
└──────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 2: API Call WITHOUT Token                              │
│  File: src/api/api.js                                        │
│                                                              │
│  const getAllProperties = async () => {                      │
│      const response = await axios.get(                       │
│          `${BASE_URL}/api/client/properties`                 │
│      );                                                      │
│      return response.data;                                   │
│  };                                                          │
│                                                              │
│  ⚠ No Authorization header (user not logged in)              │
└──────────────────────────────────────────────────────────────┘
                    │
                    │
                    │ HTTP GET Request
                    │ http://localhost:8080/api/client/properties
                    │ No auth headers
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 3: Spring Security Filter Chain                        │
│                                                              │
│   ┌──────────────────────────────────────────────────────┐   │
│   │  JwtAuthenticationFilter                             │   │
│   │  ⚠ No JWT token found                                │   │
│   │  → Skip authentication, continue                     │   │
```

```
│   └──────────────────┬──────────────────────────────────────┘   │
│                      │                                          │
│   ┌──────────────────▼──────────────────────────────────┐   │
│   │  AuthorizationFilter                                 │   │
│   │  ✓ GET /api/client/properties is permitAll()       │   │
│   │  → Allow public access                               │   │
│   └──────────────────┬──────────────────────────────────┘   │
└──────────────────────┴──────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 4: ClientController Executes                           │
│  GET /api/client/properties → ClientController               │
│                                                              │
│  @GetMapping("/properties")                                  │
│  public ResponseEntity<List<PropertyDTO>> getAllProperties() {│
│      List<PropertyDTO> properties =                          │
│          propertyService.getAllActiveProperties();           │
│      return ResponseEntity.ok(properties);                   │
│  }                                                           │
└──────────────────────┬──────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 5: Service + Repository + Database                     │
│  → SELECT * FROM properties WHERE is_active = 1              │
└──────────────────────┬──────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 6: Response with Property List                         │
│  HTTP 200 OK                                                 │
│  Body: [list of properties]                                  │
└──────────────────────┬──────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 7: React Renders Property Cards                        │
│  PublicBrowse.jsx displays property cards                    │
└──────────────────────────────────────────────────────────────┘
```

🔑 Key Architectural Patterns

1. Layered Architecture

```
Presentation Layer (React Components)
       ↓
API Layer (Axios HTTP Client)
       ↓
Controller Layer (@RestController)
```

```
        ↓
Service Layer (@Service - Business Logic)
        ↓
Repository Layer (@Repository - Data Access)
        ↓
Database (MySQL)
```

Benefits:

- **Separation of Concerns** - Each layer has specific responsibility
- **Maintainability** - Changes in one layer don't affect others
- **Testability** - Each layer can be tested independently
- **Reusability** - Services can be used by multiple controllers

---

2. DTO Pattern (Data Transfer Objects)

Why use DTOs?

- **Security** - Hide sensitive fields (passwords, internal IDs)
- **Performance** - Send only required fields over network
- **Versioning** - API contracts independent of database schema

Example Flow:

```
Database Entity (Property.java)
- id, name, address, rent, owner (entire Owner object), createdAt, updatedAt
        ↓
Service Layer converts to DTO
        ↓
PropertyDTO.java
- id, name, address, rent, ownerName (only name, not full object)
        ↓
Sent to Frontend
```

---

3. Repository Pattern

Spring Data JPA manages database interactions:

```java
public interface OwnerRepository extends JpaRepository<Owner, Long> {
    Optional<Owner> findByEmail(String email);
    List<Owner> findByIsActiveTrue();
}
```

Benefits:

- **No boilerplate SQL** - Spring auto-generates queries

- **Type-safe** - Compile-time checking
- **Consistent API** - Standard CRUD operations

🔍 Common Interview Questions & Answers

Q1: Explain the request flow from React to database

A:

1. **React Component** makes API call using Axios
2. **Axios Interceptor** adds JWT token to headers
3. **Spring Boot** receives request at Tomcat server
4. **Spring Security Filter Chain** validates JWT and checks permissions
5. **DispatcherServlet** routes to appropriate controller
6. **Controller** validates request and calls service
7. **Service Layer** implements business logic
8. **Repository** queries database using JPA
9. **Database** returns results
10. Results flow back through layers to React

Q2: How does authentication work for protected routes?

A:

- User logs in → Backend generates JWT token
- Frontend stores token in localStorage
- Axios interceptor adds token to every request
- JwtAuthenticationFilter validates token on backend
- If valid, request proceeds; if not, 401 Unauthorized

Q3: What is the role of SecurityContextHolder?

A:

- Stores authentication information for current request thread
- Each thread (request) has isolated security context
- Controllers can access authenticated user info
- Automatically cleared after request completes

Q4: How are public and protected endpoints distinguished?

A: In SecurityConfig.java:

```
.authorizeHttpRequests(authz -> authz
    .requestMatchers("/api/auth/**").permitAll()  // Public
```

```
    .requestMatchers("/api/owner/**").authenticated()  // Protected
)
```

Q5: What happens if JWT token expires?

A:

1. JwtAuthenticationFilter validates token
2. Detects expiration (ExpiredJwtException)
3. Returns 401 Unauthorized
4. Frontend intercepts 401
5. Redirects user to login page
6. Clears localStorage

Q6: How does CORS work in this project?

A:

- Frontend runs on `localhost:5173`
- Backend runs on `localhost:8080`
- Different ports = different origins
- SecurityConfig allows requests from `localhost:*`
- CORS filter adds necessary headers to responses

📊 Data Flow Summary

```
┌──────────────┐        ┌──────────────┐        ┌──────────────┐
│   React      │        │ Spring Boot  │        │    MySQL     │
│  Frontend    │        │   Backend    │        │   Database   │
│  (Port 5173)│◄──────►│  (Port 8080) │◄──────►│  (AWS RDS)   │
└──────────────┘        └──────────────┘        └──────────────┘
      │                       │                       │
      │                       │                       │
  Axios HTTP            REST API              SQL Queries
  with JWT              Controllers           (Hibernate)
                        Services
                        Repositories
```

🎯 Summary for Interviews

Key Points to Remember:

1. **React → Axios → Spring Boot → MySQL** is the full flow
2. **JWT tokens** authenticate users on every request

3. **Spring Security Filter Chain** intercepts all requests
4. **Layered architecture** separates concerns (Controller → Service → Repository)
5. **DTOs** transfer data between layers securely
6. **CORS** allows cross-origin requests between frontend and backend
7. **Public endpoints** (login, browse) don't require authentication
8. **Protected endpoints** (owner/admin APIs) require valid JWT

What makes this architecture production-ready:

- ☑ Stateless authentication (scalable)
- ☑ Role-based access control
- ☑ Secure password hashing
- ☑ Proper error handling (401, 403, 500)
- ☑ Logging and monitoring
- ☑ Database connection pooling
- ☑ RESTful API design

End of Flow & Architecture Theory Document